Intelligence artificielle pour le jeu d'Othello

Première partie — Modélisation et moteur Minimax

Auteur(s) : à compléter

Formation et groupe : à compléter

Enseignant : à compléter

Date de cette version : 29 octobre 2026

1. Introduction

Les jeux à deux joueurs constituent un cadre d'étude des méthodes de décision en intelligence artificielle. À chaque tour, un joueur doit choisir une action en tenant compte des réponses possibles de son adversaire. La difficulté ne réside donc pas seulement dans l'identification d'un coup immédiatement favorable, mais aussi dans l'anticipation de ses conséquences.

Ce projet porte sur Othello, conformément à la fiche de TP du cours de R. Mandiau, *Intelligence Artificielle : Fondements* (septembre 2026, annexe A.2, p. 323–325). Les règles permettent de représenter simplement le plateau et les actions, tandis que le nombre de suites possibles rend rapidement une exploration exhaustive jusqu'à la fin de partie trop coûteuse. Le projet vise à construire un joueur artificiel capable de choisir des coups légaux, puis à étudier les effets des algorithmes de recherche et des fonctions d'évaluation sur ses décisions et son coût de calcul.

La première étape réalisée est un moteur Minimax utilisant une évaluation par différence de pions. Il est accompagné d'une profondeur maximale, d'un budget de temps optionnel et d'une mémoire des positions. Cette version doit servir de référence pour les développements et comparaisons ultérieurs. Le présent rapport explique la représentation retenue, les adaptations de Minimax aux règles d'Othello et les choix de conception de cette première implémentation.

2. Analyse et modélisation du jeu

2.1. Règles prises en compte

Le plateau comporte 64 cases disposées sur une grille de 8 × 8. Deux joueurs possèdent des pions de couleurs opposées. Un coup consiste à poser un pion sur une case vide de manière à encadrer au moins un pion adverse entre le nouveau pion et un pion de sa propre couleur. Les pions ainsi encadrés sont retournés. La recherche des captures s'effectue dans les huit directions du plateau.

Un joueur sans coup légal passe son tour. La partie se termine lorsque les deux joueurs ne peuvent plus jouer, même si des cases restent vides. Le joueur ayant le plus de pions remporte la partie ; des nombres égaux correspondent à un match nul. Dans ce projet, les blancs commencent, conformément à la convention du sujet (fiche de TP, p. 323).

2.2. Représentation d'un état

La classe `Board` représente la grille par un tableau NumPy d'entiers. Les valeurs retenues sont 0 pour une case vide, 1 pour un pion noir et -1 pour un pion blanc. Cette convention permet d'obtenir la couleur adverse en changeant le signe de la couleur courante.

Un coup est un couple d'indices `(ligne, colonne)` commençant à zéro. La valeur `None` représente une absence de coup. Pour décrire complètement une situation de jeu, il faut associer la grille à la couleur du joueur dont c'est le tour. La même disposition de pions ne définit pas nécessairement les mêmes actions si le joueur courant change.

La méthode `get_flips()` détermine les captures possibles sans modifier la grille. Un coup est légal si cette liste n'est pas vide. La méthode `apply_move()` pose le pion et effectue les retournements. Enfin, `copy()` produit une copie indépendante utilisée pour simuler les coups sans altérer la position réelle.

2.3. Organisation du programme et état d'avancement

Le programme sépare les règles du jeu, les joueurs, l'affichage et la recherche. Le tableau 1 présente les principales responsabilités.

Tableau 1 — Organisation du projet au stade de cette rédaction.

Module	Responsabilité et état
`constants.py`	Couleurs, taille, directions et types de coups
`board.py`	Grille, règles de capture, coups légaux, copies, score et fin de partie
`player.py`	Joueurs humain, aléatoire et IA ; l'IA possède son moteur Minimax
`minmax.py`	Évaluation, recherche, délai, mémoire et statistiques
`view.py`	Interface console séparée de la recherche
`game.py`	Structure de gestion d'une partie ; méthodes de déroulement encore à compléter
`main.py`	Configuration actuelle humain contre aléatoire ; intégration de la partie complète à finaliser
`test_minmax.py`	Dix tests ciblés du moteur de recherche
`test_board.py`	Scénarios de tests du plateau déclarés mais encore non implémentés

L'existence d'un moteur de recherche utilisable avec `Board` ne signifie donc pas que le lancement interactif par `main.py` est terminé. Le rapport distingue la validation de ce moteur de l'intégration complète de l'application.

3. Conception du moteur Minimax

3.1. Fonction d'évaluation

La première stratégie implémentée est l'évaluation absolue proposée dans le sujet (fiche de TP, p. 324). Pour une position s et la couleur de l'IA c, elle est définie par la différence :

E(s, c) = nombre de pions de c − nombre de pions de la couleur adverse.

Un score positif indique une avance en pions et un score négatif un retard. Cette valeur reste calculée du point de vue de l'IA, y compris lorsque la recherche simule le tour adverse. Elle fournit une estimation simple aux feuilles de l'arbre lorsque la profondeur maximale est atteinte. Elle ne garantit pas que le joueur possédant le plus de pions pendant la partie remportera celle-ci.

Les positions terminales sont traitées séparément : une victoire de l'IA reçoit la valeur +∞, une défaite -∞ et une égalité 0. Cette convention donne priorité à une victoire connue sur une simple avance provisoire. Elle ne distingue pas les victoires selon leur écart de pions. Les infinis servent à comparer les résultats ; c'est le test de fin de partie qui établit que le résultat est définitif.

3.2. Alternance MAX et MIN

Minimax suppose que chaque joueur sélectionne la meilleure action selon le critère retenu. Aux niveaux correspondant à l'IA, la recherche conserve la valeur maximale des successeurs. Aux niveaux adverses, elle conserve leur valeur minimale, puisque les scores restent exprimés du point de vue de l'IA.

Par exemple, si un coup A permet à l'adversaire de choisir entre des valeurs 3 et 5, sa valeur est 3. Si un coup B autorise des réponses de valeurs -2 et 8, sa valeur est -2. L'IA préfère A : elle anticipe la réponse adverse la plus défavorable selon son évaluation, au lieu de supposer que l'adversaire lui offrira 8.

La méthode `_minimax()` calcule la valeur d'une position. La méthode `_best_move()` compare les coups disponibles à la racine et conserve les coordonnées associées à la meilleure valeur. En cas d'égalité, le premier coup dans l'ordre de parcours du plateau est conservé. La recherche est ainsi déterministe à profondeur fixe.

3.3. Conditions d'arrêt, profondeur et passes

La recherche vérifie d'abord si la partie est terminée, puis si la profondeur restante est nulle. Cette priorité garantit qu'une position terminale atteinte exactement à la limite reçoit un résultat définitif plutôt qu'une simple estimation.

Dans le code, un niveau correspond à une action individuelle, ou demi-coup. La profondeur est diminuée après chaque coup simulé. Le sujet définit une profondeur de 1 comme une action du joueur et une réponse adverse (fiche de TP, p. 324) : la conversion retenue est donc **profondeur du code = 2 × profondeur du TP**. Cette unité doit accompagner toutes les mesures expérimentales.

Lorsqu'un seul joueur est bloqué, la recherche continue avec son adversaire sur la même grille. Nous choisissons de ne pas consommer de profondeur pour une passe, puisqu'aucun pion n'est posé. Si les deux joueurs sont bloqués, le test terminal arrête la recherche. Ces conventions adaptent le raisonnement général de Minimax aux règles d'Othello et doivent rester identiques dans les futures versions.

3.4. Coût de la recherche

Pour un facteur de branchement moyen b et une profondeur d, le nombre de feuilles peut croître comme b puissance d. Le facteur réel varie selon la position. Cette croissance motive la limitation de profondeur et de temps, ainsi que la mémorisation. La version actuelle ne réalise aucune coupure Alpha-Bêta.

Le parcours en profondeur crée des copies de plateau pour explorer les successeurs. L'arbre entier n'est pas conservé explicitement : la mémoire persistante stocke plutôt les valeurs de sous-problèmes identifiés par leurs positions et leurs paramètres de recherche.